

WEB-BASED VEHICLE AUCTION PLATFORM

**K.K.J. HAMID
R222226**

JULY - 2026



WEB-BASED VEHICLE AUCTION PLATFORM

HAMID K.K.J.
R222226

Supervisor: Dr. Subodha Hettiarachchi Gunawardena

JULY - 2026



**This dissertation is submitted in partial fulfilment of the requirement of the
Degree of Bachelor of Information Technology (External) of the
University of Colombo School of Computing**

Declaration

I certify that this dissertation does not incorporate, without acknowledgement, any material previously submitted for a degree or diploma in any University/Institute, and to the best of my knowledge and belief, it does not contain any material previously published or written by another person or myself except where due reference is made in the text.

Signature of Candidate:

Date:.....

Name of Candidate:

Countersigned by:

Signature of Supervisor(s)/Advisor(s):

Date:.....

Name(s) of Supervisor(s)/Advisor(s):

Abstract

TaproJapan Co. Ltd., a Japan-based vehicle exporter, managed its vehicle auctions through WhatsApp broadcasts and informal messaging. This process offered no audit trail, no buyer verification, and frequent disputes over who placed the highest bid, creating a scaling bottleneck as the client's international buyer base grew. This project designed and implemented a web-based vehicle auction platform to replace this manual workflow with a structured, transparent, and auditable digital system.

The system was built as a three-tier full-stack application: a React 19 single-page front end, a layered Node.js/Express REST API, and a PostgreSQL relational database. The backend separates routes, controllers, services, and repositories, with all business logic such as authentication, role-based access control, bid validation, and auction status transitions concentrated in the service layer. The database schema evolved through eight incremental migrations, moving from a basic four-table design to a normalized structure supporting auction-scoped bidding, hidden reserve prices, and a three-state buyer verification workflow.

Key functionality delivered includes JWT-based stateless authentication, an admin-only buyer verification workflow, a vehicle catalogue with multi-criteria search and filtering, a time-bound auction module with dual-layer status correction, minimum-increment bid validation, reserve-price privacy protection for non-admin users, and an admin dashboard for vehicle, auction, and user management with manual or automatic winner selection. Direct-to-Cloudinary image uploads keep the backend stateless with respect to binary data.

The system was verified through unit and integration tests covering authentication, vehicle CRUD, bidding rules, and auction lifecycle transitions, supplemented by manual end-to-end and User Acceptance Testing with the client. Testing surfaced and resolved several non-trivial defects, including an authentication flow that blocked all logins, a role-escalation vulnerability in registration, and an architectural flaw that scoped bids to vehicles rather than auctions. The resulting platform meets its core functional and non-functional requirements and gives the client a scalable, transparent replacement for its manual auction process, with payment integration and shipping logistics identified as natural extensions for future work.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Subodha Hettiarachchi Gunawardena, for his guidance, technical feedback, and continued support throughout the duration of this project. I am also grateful to the staff of the University of Colombo School of Computing for the resources and environment that made this work possible. My thanks also go to TaproJapan Co. Ltd. for their cooperation as the client of this project, for sharing their business process in detail, and for their feedback during user acceptance testing.

Table of Contents

When preparing a table of contents, use only two levels, list the main chapters as first-level headings (e.g., Introduction, Design, Implementation) and include important subheadings as second-level entries (e.g., 2.1 System Architecture, 2.2 Tools Used). Align page numbers neatly and ensure the numbering matches the actual document. To maintain consistency and accuracy, it is recommended to use the built-in table of contents and heading styles feature in your word processing application (such as Microsoft Word or Google Docs). This allows you to automatically generate and update the table of contents based on the heading levels used in your document. Ensure proper formatting, align page numbers neatly, and keep the layout clean and easy to navigate.

Table of Contents

Declaration	I
Abstract	II
Acknowledgements	III
Table of Contents	IV
List of Figures	V
List of Tables.....	VI
List of Acronyms.....	VII
Chapter 1 Introduction.....	1
Chapter 2 – Analysis.....	2
2.1 Analysis	3
Chapter 3 – Design.....	4
Chapter 4 – Implementation	6
Chapter 5 – Evaluation.....	8
Chapter 6 – Conclusion	9
References.....	10
Appendices.....	11
General Guidelines.....	12

List of Figures

All figures in the dissertation should be numbered and named using an appropriate caption. Numbering is done using chapter number and a sequence number (e.g. Figure 3.2 for second figure in Chapter 3). Figures in the appendices are numbered using the Appendix letter (e.g. Figure C.2 for second figure in Appendix C). List of figures consists of figure number, captions and page numbers. List can be generated using features of a word processing package. All figures used in the main chapters must be described in text prior to its use and must be referred to using its figure number. For example, in Section 3.5 of this document Figure 3.1 is referred to in text in the paragraph before the figure.

Figure 1.1: Sample Figure..... 1

List of Tables

All tables in the dissertation should be numbered and named using an appropriate caption. Numbering is done using chapter number and a sequence number (e.g. Table 3.2 for second table in Chapter 3). Tables in the appendices are numbered using the Appendix letter (e.g. Table C.2 for second table in Appendix C). List of tables consists of table number, captions and page numbers. List can be generated using features of a word processing package. All tables used in the main chapters must be described in text prior to its use and must be referred to using its table number. Example: In section 5.1 of this document Table 5.2 is referred to in text in the paragraph.

Table 1.1: Sample Table 2

List of Acronyms

API – Application Programming Interface

CRUD – Create, Read, Update, Delete

CDN – Content Delivery Network

CRC – Cyclic Redundancy Check (not used; placeholder removed)

ERD – Entity-Relationship Diagram

HTTP – Hypertext Transfer Protocol

JWT – JSON Web Token

ORM – Object-Relational Mapping

RBAC – Role-Based Access Control

REST – Representational State Transfer

SDLC – Software Development Life Cycle

SPA – Single Page Application

SQL – Structured Query Language

UAT – User Acceptance Testing

UI – User Interface

UUID – Universally Unique Identifier

Chapter 1 Introduction

.1 Problem and Background

are included in assessment criteria. Typical subsections include:

- Problem and background;
- Motivation;
- Aims, and Objectives of the Project; and
- Scope of the Project.

When writing the scope of your work, it's important to differentiate it from simply listing features or functional requirements. The scope defines the **overall boundaries** of the project. It explains what the system will cover, the target users, the environment it will operate in, and the type of problems it is intended to solve. It gives a high-level understanding of what the project includes and excludes. In contrast, a list of features or functional requirements is a detailed, technical breakdown you have to write in the analysis chapter. While the scope may mention major features as part of its explanation, it should focus more on the **purpose, coverage, and limitations** of the project rather than itemizing every specific function. This distinction helps keep the introduction conceptual and avoids unnecessary detail early in the dissertation.

Further, provide an outline of the thesis starting from chapter 2. These descriptions should be very brief, with one or two sentences for each chapter.

This chapter may be only a few pages in length, but it is one of the most important chapters of the thesis. It will shape the expectations of the reader, and provide them with a guide to the rest of the work.

Chapter 2 – Analysis

In the Analysis chapter of your software implementation project, the focus is on thoroughly understanding and defining the problem and system requirements before starting design or development. Begin by describing the current system and clearly outlining existing challenges or inefficiencies. Use flowcharts to illustrate the existing processes and point where issues arise, providing a clear picture of the problem and context.

Next, review similar systems, referencing published studies, software tools, or case examples that have addressed comparable problems. This review supports your design choices by showing how others have successfully approached similar challenges, establishing a foundation for your proposed solution. Following this, clearly identify and prioritize the functional and non-functional requirements of your system. Functional requirements define what the system must do, such as user authentication or data processing, while non-functional requirements cover performance, usability, and reliability.

Importantly, security should be included as a critical functional requirement because modern systems depend on various security controls to operate safely and effectively. These include access controls, data encryption, audit logging, and input validation, all of which are essential functions that protect the system's integrity, confidentiality, and availability. Treating security as a core functional requirement ensures it is integrated into the system from the beginning rather than as an afterthought.

Finally, justify the choice of your Software Development Life Cycle (SDLC) model by explaining why it best fits your project's scope and complexity by minimizing the implementation defects and risks. Also, provide reasoning for your selection of programming languages, databases, and other technologies based on their relevance, efficiency, and ability to meet your project's needs. This chapter demonstrates that your system design is grounded in a clear understanding of the problem, requirements, and practical solutions.

Typical subsections include:

2.1. Analysis

- 2.1.1. Existing System and Problem Description (Describe how the existing system or manual process works. Explain the limitations, errors, bottlenecks, or risks in the current system. Include one or more flowcharts to visually demonstrate the current process and highlight where issues occur.);
- 2.1.2. Review of Similar Systems (Briefly describe 2–3 similar systems or solutions. Include a feature comparison table.);
- 2.1.3. System Requirements Analysis; and
 - 2.1.3.1. Functional Requirements (List and explain the core features your system must provide);
 - 2.1.3.2. Security as a Functional Requirement (Justify why access control, encryption, input validation, etc., are essential functions for secure system operation.); and
 - 2.1.3.3. Non-Functional Requirements (Specify quality attributes like performance, reliability, scalability, usability, and maintainability.).
- 2.1.4. Justification of Development Approach and Technologies.
 - 2.1.4.1. Chosen SDLC Model and Rationale (Explain which SDLC model (e.g., Waterfall, Iterative, etc.) you selected and why it's suitable for your project in terms of scope, complexity, and risk control.); and
 - 2.1.4.2. Technology Stack Justification (Justify your choice of programming languages, databases, and tools based on performance, relevance, team expertise, or technical requirements.).

Chapter 3 – Design

In the Design chapter of your software implementation project, the objective is to convert the system requirements into a structured and detailed plan for implementation. This involves defining the system architecture, internal module behavior, data models, and user interfaces, with all design elements clearly traceable to the requirements established in the Analysis chapter. Begin by outlining the overall system architecture using diagrams such as layered architecture diagrams or deployment diagrams. These help visualize how the core components—user interface, application logic, database, and external systems—interact within the system.

For internal component design, the approach will depend on whether you are using object-oriented or non-object-oriented design. In object-oriented systems, use case diagrams, class diagrams, sequence diagrams, and activity diagrams should be used to describe user goals, object structures, interactions, and process flows. In non-object-oriented systems, while use case diagrams can still illustrate system functionalities, you should also use data flow diagrams (DFDs), structure charts, and flowcharts to represent the flow of data and control between functional modules. Adapted sequence diagrams may be used to show the temporal interaction between functions or system components, and activity diagrams remain applicable in both paradigms to model workflows and system behavior.

Data modeling is a critical part of the design chapter. Use Entity-Relationship Diagrams (ERDs) to describe the structure of the system's database, detailing entities, attributes, relationships, and constraints. Discuss normalization, data integrity rules, and key constraints to justify your data model. For user interface design, present wireframes or mockups of key screens and justify your design decisions based on usability, accessibility, and responsiveness. Mention design principles like consistency and simplicity, and any tools used to develop the interface layout.

Throughout the chapter, explain the design methodologies and principles that guide your system architecture and component design—such as modularity, reusability, and separation of concerns. Specify any design patterns or architectural styles employed, like MVC or layered architecture, and explain their suitability to the system's goals. Finally, provide a mapping between each system requirement and the corresponding design

elements, ensuring that the design comprehensively addresses all user and system needs defined earlier.

Suggested subsections for the chapter could include:

1. Introduction to System Design;
2. Design Principles and Methodologies;
3. System Architecture Overview;
4. Component and Module Design;
5. Workflow and Behavioral Modeling;
6. Data Modeling; and
7. User Interface Design.

Chapter 4 – Implementation

In the Implementation chapter of your dissertation, the focus shifts from design plans to the actual development of the system. This chapter should document how the system was built, including the development environment, coding practices, module structure, important algorithms, and version control mechanisms. It should provide enough technical detail to demonstrate how the design was realized in code, ensuring consistency with the design specifications from the previous chapter.

Start by describing the development and deployment environments used throughout the project. Mention the operating system, programming languages, IDEs, libraries, databases, and any other relevant tools or platforms used. This sets the context for reproducibility and technical compatibility.

Next, present the development practices adopted during implementation. Outline how the system was broken into logical modules or components, and briefly describe the responsibilities and interdependencies of each. Discuss your coding standards, including naming conventions, file organization, and formatting rules. Highlight how you ensured readability, maintainability, and documentation through consistent use of inline comments and structured documentation headers. If code linters, formatters, or static analysis tools were used, mention them here.

Highlight the critical code segments that form the core functionality of the system. These may include authentication logic, key algorithm implementations, data validation routines, or system workflows. Provide annotated code snippets (screenshots with black text (or different colors to show syntax) in white background) where appropriate and explain how they fulfill specific design or functional requirements. Make sure to not to include too much code. Collectively, all the code snippets should not exceed two pages. Where performance or efficiency matters, describe the data structures and algorithms you selected, including their rationale (e.g., time complexity, memory efficiency, scalability). You may also mention how security or error-handling logic is implemented within these sections.

Software development often involves iterative updates to the code. Describe your software version management practices, such as using Git for source code control.

Include how you structured your repositories (e.g., branches, commit messages, tags) and whether you used any platforms like GitHub, GitLab, or Bitbucket. If relevant, mention tools or practices for issue tracking, code reviews, or automated testing.

Throughout the chapter, connect your implementation decisions back to the design specifications and system requirements to show traceability and completeness. This chapter should clearly reflect the translation of plans into working code, emphasizing technical depth, adherence to standards, and maintainability.

Chapter 5 – Evaluation

In the Evaluation chapter of your software implementation project, your goal is to demonstrate that the system was thoroughly tested, validated, and accepted by its intended users. This chapter should provide clear evidence that the system meets both its technical and user-defined requirements, and that it performs reliably in real-world scenarios.

Begin by describing the testing strategy you followed to verify and validate the system at different levels. Start with unit testing, where individual functions or modules were tested in isolation to confirm they worked as intended. Mention the tools you used for writing and executing these tests, and include representative test cases or screenshots of test outputs. Then explain your approach to integration testing, showing how components were combined and tested together. Describe how you ensured that data flowed correctly between modules, and note any errors or mismatches that were identified and resolved.

System testing followed once all components were integrated. This stage involves evaluating the system as a whole, in conditions that resemble the real usage environment. Provide examples of full-system test scenarios that reflect the key use cases. For each case, explain the inputs, expected outputs, and actual results. Highlight any performance issues, functional errors, or usability concerns found during this process and how they were fixed. Where appropriate, include logs, test reports, or screen captures as proof of test execution.

In addition to technical testing, user feedback is essential. Describe how you conducted User Acceptance Testing (UAT), where actual users or representatives from your target audience interacted with the system. Explain how the testing sessions were organized, what tasks users were asked to perform, and how you gathered feedback (e.g., through interviews, surveys, or observation). Summarize the feedback received, including both positive responses and suggested improvements. Emphasize any design changes or feature adjustments you made in response to this feedback to enhance usability or user satisfaction.

Make sure collectively the space consumed by the screenshots has not exceed 2 pages.

Chapter 6 – Conclusion

In the Conclusion chapter of your software implementation project, the objective is to reflect on the overall journey of the project, from problem identification to system delivery, and evaluate its success and impact. This chapter should provide a critical assessment of what was achieved, what challenges were encountered, and how effectively the original goals were met. It should not simply summarize previous chapters, but rather synthesize your findings and insights, demonstrating your understanding of the technical, practical, and personal aspects of the project.

Begin by critically evaluating the effectiveness of your solution. Discuss whether the system met the core functional and non-functional requirements, and how well it addressed the original problem. Consider the design decisions, technologies used, and development process highlighting what worked well and where limitations were encountered. Mention any trade-offs or compromises made during implementation and justify them. This is your opportunity to show that you can assess your own work objectively, identifying strengths and acknowledging weaknesses.

Next, include a personal reflection on the learning experience gained through the project. Reflect on how the work contributed to your understanding of the module's Intended Learning Outcomes (ILOs), such as applying software development principles, working independently, solving technical problems, and engaging in critical thinking. Mention specific skills you developed or improved, such as time management, use of development tools, communication, or documentation. Be honest and constructive; this is not about listing achievements but about recognizing growth and areas for further development.

Finally, discuss potential future work. Identify limitations in your current system that could be addressed in the next phase or by another developer. Suggest enhancements or new features that could improve functionality, scalability, or user experience. You may also reflect on how emerging technologies or alternative design approaches could benefit the system.

This chapter is your opportunity to wrap up your project with insight and maturity. It should leave the reader with a clear understanding of what was achieved, how it was evaluated, what you learned, and how the work could be extended from where you left off.

References

It is very important to acknowledge any of the work of others that the candidate used or adapted in the project, or that provided the essential background or context to the dissertation. Please note that IEEE is the recommended referencing and citation style for your dissertation. It is also recommended that the candidate use a reference management tool such as Mendeley or Zotero

Following examples show how the referencing should be done.

Example 1:

Systems analysis and design techniques are considered essential for developing client/server and web-centric applications [6, 7].

Example 2:

Software testing [8] is an iterative process.

Example 3:

“Plagiarism is an act of fraud. It involves both stealing someone else's work and lying about it afterward” [9].

In the **References** section, each citation should be listed in the relevant format (Refer to an IEEE referencing and citation style guide). For example, the reference section entries for the above three examples would be;

- [6] J.L. Whitten and L.D. Bentley, *Systems Analysis and Design Methods*, 7th ed. Tata McGraw-Hill, 2007.
- [7] UCSC, *The Virtual Learning Environment for the BIT Students*, 2006. [Online] Available: <http://vle.bit.lk> , [Accessed: 30 Oct, 2013]
- [8] I. Sommerville, *Software Engineering*, 8th ed. Addison-Wesley, 2006.
- [9] Plagiarism.org - Best Practices for Ensuring Originality in Written Work, "What is Plagiarism?", 2015. [Online]. Available: <http://www.plagiarism.org/plagiarism-101/what-is-plagiarism/>. [Accessed: 29- Dec- 2015].

Appendices

The appendices include further information that is not essential to be included in the main text, but nevertheless could be useful to interested readers. The following appendices should be included in the dissertation:

Appendix A - System Manual

Technical documentation is included here. These details should guide candidates who wish to continue or use your project work and allow amendments and extensions to the code. Provide program installation, compilation, and execution details. Documentation should point to locations where DLL's and reusable code can be found if they are publicly available.

Appendix B - User Manual

May include thorough and comprehensive documentation at a level, which is appropriate to the identified users. User documentation may cover all aspects of the system, with appropriate screenshots and explanations. **Failing to include such documents means that you have failed to implement a critical component of the system and it could result in not calling for project evaluation.**

Appendix C - Management Reports

In addition to producing day to day transaction reports (e.g. a payroll system should produce an individual pay sheet, coin analysis to make cash payments, EPF report, etc.) a system must produce summarised reports for the management (e.g. monthly, quarterly payments made by organisation, employees, overtime hours by employee, etc.). These reports should be included here. The usefulness of the system will be judged using these reports. **Failing to include such reports means that the candidate had failed to achieve his objectives and it could result in not calling for project evaluation.** Ensure that the reports contain meaningful information which could be obtained through your system using a sufficient amount of data.

General Guidelines

Candidates are strongly advised, while writing is in progress, to show each chapter to their supervisors for necessary feedback especially on technical content. Please follow the instructions below to minimize correction time.

- The candidates are advised to follow the typing recommendations given below to typeset their dissertations.
 - Dissertation text Times New Roman - 12pt
 - Text in tables and code listing - 11pt
 - Line spacing (preface and main text) - 1.5
 - Line spacing appendices - 1.0
 - Left margin - 37mm
 - Top/bottom/right margins - 25mm
 - Chapter heading - 24pt bold
 - Section headings - 16pt
 - Subsection headings - 14pt
 - Other headings - 12pt bold
 - Tables headings font - 11pt bold
- All pages should be numbered including Chapter 1 beginning from page 1. Use roman numerals for front matter as used in this guidelines document.
- **Plagiarism** is the presentation of another person's thoughts or words as though they were the candidate's own. The candidate should avoid this when writing his dissertation. All sentences or passages quoted in reports from other people's work have to be specially acknowledged by clear cross-referencing to author, work and page(s). Direct quotations from published or unpublished work of others should always be clearly identified as such by being placed inside quotation marks, and full reference to their source should be provided in the proper form. Equally, if another person's ideas or judgements are summarized, the candidate should refer to that person in the main text of the dissertation, and include the work referred to in the references section of the dissertation. Failure to adhere to these rules may result

in an allegation of cheating. **All suspected cheating will be reported as examination offenses.** Any illustrations, which are not the work of the candidate can be used only with the explicit permission of the originator and should be specially acknowledged. The project is an important component of the degree and plagiarism in project work is taken very seriously, and when discovered will imply severe penalties and consequences for the culprit's degree and possibly for his entire future career. Such a candidate will fail the project and the degree examination as a whole when plagiarism in project work is discovered. Candidates will not be allowed to repeat the project and other degree components for a specified number of years depending on the decision by the university. Therefore, it is important to give credit where it is due and acknowledge all work borrowed and emphasize what the candidate's distinct contribution has been in the project. **Similarity Index should be less than 20% to avoid consideration for possible plagiarism attempts.**

Writing Style

The following are some of the general points you should use them to improve writing style:

- Try to minimize mixing of unrelated issues in the same chapter. For example, in the design chapter, do not discuss testing issues. One exception to this rule would be when arguments need to justify actions related to the current topic.
- Throughout the dissertation, use standard software engineering or computer science terminology. (refer Guide to the Software Engineering Body of Knowledge Version 3.0 SWEBOK®,)
- Use a consistent naming convention. Do not refer to the same concept or “thing” by different names in different parts of the dissertation.
- Write short, focused, and coherent paragraphs. Each paragraph should have a clearly stated topic sentence that describes the main issue/context covered in the paragraph. The rest of the sentences should provide discussions on the topic sentence, or answer the question raised in the topic sentence.

- Do a grammar and style check before submission. It is ESSENTIAL that you proofread your dissertation manually before printing the final copy for submission.
- Omit needless words—redundant modifiers, pompous diction, excessive detail.
- Eliminate "narration," expressions such as "It is my opinion that," "I have concluded," "the main point supporting my view concerns," or "certainly there is little doubt as to. ". Focus attention solely on what the reader needs to know.